On the Exploratory Testing of Mobile Apps

Mariana Souza*
Federal University of Technology - Parana (UTFPR)
souza.2012@alunos.utfpr.edu.br

Isabel K. Villanes
Federal University of Amazonas (UFAM)
isabelkarina@icomp.ufam.edu.br

Arilo Claudio Dias-Neto
Federal University of Amazonas (UFAM)
arilo@icomp.ufam.edu.br

Andre T. Endo
Federal University of Technology - Parana (UTFPR)
andreendo@utfpr.edu.br

ABSTRACT

While the literature acknowledges that mobile apps present different testing challenges and automated solutions have been pursued, it lacks a better understanding of how pervasive practices of manual testing (namely Exploratory Testing - ET) can be more effectively applied. This paper aims to investigate the use of ET in mobile apps. With this study, we intend to have a better understanding of how exploratory testing is employed, its effectiveness, and its usage in an ample and diverse range of apps. To do so, we conducted two studies. The first study was conducted for the purpose of applying ET to apps with diverse contexts and available on Google Play in order to analyze whether testers actually explore all possible scenarios that apps may display. The second study, also applied the ET, however in two apps that were developed by a software development company; this study has the objective of applying the ET in order to identify bugs of different levels, that often cannot be revealed using other techniques. As expected the first study revealed that there are several test scenarios that are not exploited by the testers, yet the 40 participants revealed on average 5 bugs in 1.5h of test sessions. The second study revealed 64 bugs and 21 issues in two apps. Such revealed bugs are of different criticality and category. ET has shown to be a promising technique to uncover bugs, though test professionals can be better guided to explore their apps and search for bugs in scenarios related to mobile specific events.

CCS CONCEPTS

• **Software and its engineering → Software verification and validation**.

KEYWORDS

Exploratory Testing, Android, Mobile Applications, Manual Tests

ACM Reference Format:
Mariana Souza, Isabel K. Villanes, Arilo Claudio Dias-Neto, and Andre T. Endo. 2019. On the Exploratory Testing of Mobile Apps. In *IV Brazilian Symposium on Systematic and Automated Software Testing (SAST 2019), September 23–27, 2019, Salvador, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3356317.3356322

SAST 2019, September 23–27, 2019, Salvador, Brazil

ACM ISBN 978-1-4503-7648-8/19/09…$15.00
https://doi.org/10.1145/3356317.3356322

1 INTRODUCTION

From the 1990s, mobile devices began to emerge, such as tablets, smartphones, wearables and e-readers, making it easy to access remote communications. Thus, different services have also appeared for all types of users, aiming to make life easier for all. Mobile computing is a paradigm that allows users to access information services regardless of their location, which involves mobility, wireless communication and processing [22]. This environment brought a wide variety of mobile applications, known as apps. Such apps are software designed to be installed and run on mobile devices, such as on a smartphone [26]. Mobile apps are part of the lives of billions of people around the world, they have changed people's routines and behavior and are used on a large scale and for several hours of the day [2]. In recent years there has been a considerable growth of mobile device users, which has led to an increase in the number of apps for such devices [18].

Due to the growth of mobile devices, especially smartphones, apps have also seen an excessive increase in recent years, posing several challenges in their reliability. Therefore, it is necessary that test techniques be applied to them [18]. To reduce the incidence of problems or defects in such applications, tests are applied, which are typically performed during the Software Engineering process. Verification, Validation and Testing (VV & T) are techniques that are performed to reduce the presence of defects that may occur in software development; these techniques are applied to guarantee the quality of the software [19, 20]. The software testing aims to verify and validate, during the execution of the software, if it is behaving as expected. However, software testing is not a simple task due to attributes of each software. Therefore, software testing is not always considered fundamental for professionals who create apps, often this step is done incompletely or erroneously [11]. Most of the tests applied in the industry are manual [11], however the literature addresses much more automated testing [29].

Among the existing techniques of manual testing we can highlight *Exploratory Testing* (ET) [3, 8, 25, 27]. ET is a powerful and efficient way to test software manually, integrating design, execution and test analysis. ET [13] is one of the most used test approaches in Software Engineering. The use of ET can bring several benefits, both for testers and for identifying defects. ET helps to expand the tester's imagination and intuition to find unexpected defects. Thus, by exploiting the software, the tester performs more and more test

cases, improving productivity and consequently involves a research process that helps to find more errors than normal tests [8, 16, 25].

Another benefit that the ET makes possible is to reduce the documentation overhead [3, 8, 16, 25], since it does not require scripting. There is a concurrent learning of the tester [3, 10], because the more he explores his creativity using ET, the more knowledge and experience he gets by generating new ideas during test execution.

Therefore this work aims to investigate the use of ET in mobile apps. With this study, we intend to have a better understanding of how exploratory testing is employed, its effectiveness, and its usage in an ample and diverse range of apps. This paper is organized as follows. Section 2 describe background concepts. Section 3 specifies some related works. Section 4 presents the research methodology and the two studies. Section 5 explain the analysis of results. Section 6 includes a discussion and lessons learned. Finally, Section 7 highlights some conclusions.

2 BACKGROUND

This section introduces common concepts of mobile app testing, the exploratory testing and the Session Based Testing technique.

2.1 Mobile App Testing

Mobile apps are software designed to be installed and run on mobile devices, such as on a smartphone [26], to operate on Mobile OS platforms (e.g. Android, iOS). This software may be written in various programming languages (e.g. Java, Kotlin, Objective-C). The purpose of mobile application is to enhance user's daily life throughout [17].

Initially mobile apps were developed for entertainment, but there are several apps that care about important issues like health, economy, information and even personal safety [26]. Mobile apps running in mobile devices are becoming so popular that they are easy to download and install. Since the first quarter of 2019 Android users are able to choose between 2.1 million apps and Apple users between 1.8 million available apps [24].

The term Mobile Testing refers to "testing activities for native and web applications on mobile devices using well-defined software testing methods and tools to ensure quality in functions, behaviors, performance, and quality of service, as well as features, such as mobility, usability, interoperability, connectivity, security, and privacy" [4]. A challenge, among several that faces the mobile app testing, is the diversity of devices, known as Fragmentation, each with several versions of OS, screen sizes, resources, etc. These characteristics also represent a challenge for ET. As ET is a manual testing, testing Apps on a large number of devices requires more time, human and financial resources which increases the cost of testing [6].

2.2 Exploratory Testing (ET)

The ET was introduced by Kaner [13] ET is an approach to test software without pre-designed test cases. ET is typically defined as simultaneous learning, test design and test execution [3, 12]. Figure 1 shows the ET cycle representing that at the same time that the tester plans, tests, performs, and analyzes the test, it also gains in learning. The starting point of this process is planning.

ET is a technique widely used by industrial practitioners [9]. Some pros of this technique are the tester's ability to explore his creativity, using his practices and abilities, having less dependence on documentation. A disadvantage is that testers can spend a lot of time testing a single feature, or testing the same situations without encountering bugs. Lack of preparation, structure and orientation can lead to unproductive hours and repetition of the same functionality [28].

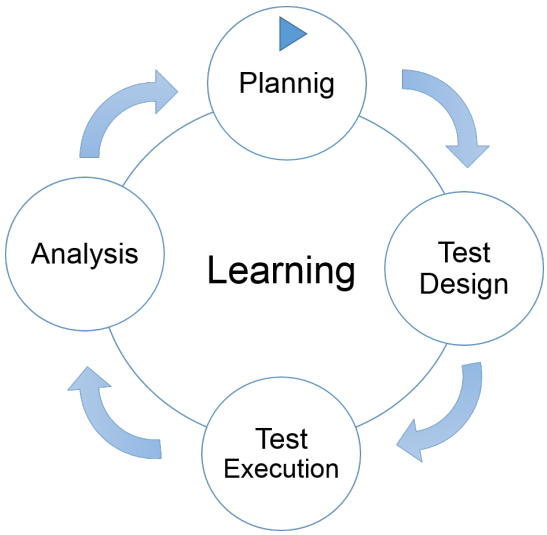

Figure 1: Exploratory Testing cycle.

Recent studies use ET to support other testing techniques such as model-testing [5] and GUI testing [21]. ET is sometimes conducted using session-based testing to structure the activity. In session-based testing, ET is conducted within a defined time-box, and the tester uses a test charter containing test objectives to guide the testing.

2.3 Session Based Testing (SBT)

SBT is an alternative framework for doing ET based on time. SBT is a technique that manages tests delimiting session time. Each session will contain limited functionalities that should be tested. There is not a certain time for each test session, but Itkonen states that test time generally ranges from a few hours to a maximum of 12 hours [7]. In each session a number of charters are distributed.

The charter is the main point / goal of the session. The charter can include a set of features, which should be explored in a session. The charters are created by the team of testers before starting the testing sessions [3].

The structure of SBT is as follows: over a period of time the tester interacts with the system to perform a test mission and report results. SBT have some basic elements such as the time, system, mission, tester and report. There is no script to follow to perform test sessions, so ET can be applied in the concept of SBT. However, the tester himself must plan and manage his efforts by testing each session [7].

3 RELATED WORK

Itkonen and Rautiainen (2005) conducted an interview with seven professionals in three companies. The results show that the main

reasons for using ET in companies were the difficulties in designing test cases for complicated functionalities. Results found greater efficiency for detection of defects using ET. While the difficulty was to manage the coverage of tests [10].

Itkonen et al. (2007) presents a controlled experiment comparing the defect detection efficiency of ET and test case based testing (TCT). They conducted an the experiment, with 79 advanced software engineering students performed manual functional testing on an open-source application with actual and seeded defects. Each student participated in two 90-minute controlled sessions, using ET in one and TCT in the other. They found no significant differences in defect detection efficiency between TCT and ET. However, TCT produced significantly more false defect reports than ET [8].

Joorabchi et al. (2013) conducted a study to understand the current challenges in mobile app development. They did an exploratory study setting the goal of discovering the process and challenges of mobile app development on different platforms. To do so, they conducted and analyzed interviews with 12 senior app developers, from nine different manufacturing companies, experts on platforms like iOS, Android, Windows Mobile / Phone and Blackberry. Based on the interviews' results, they created and distributed an online survey, which was completed by 188 mobile app developers around the world. The results reveal lack of robust monitoring, analysis, and testing tools and the prevalence for manual testing [11].

Kochhar et al. (2015) examined the current state of testing of apps, the tools that are commonly used by app developers, and the problems faced by them. To get an insight on the test automation culture, they conduct two different studies. In the first study, they analyse over 600 Android apps collected from F-Droid. They checked for the presence of test cases and calculate code coverage to measure the adequacy of testing in these apps. They also survey developers who have hosted their applications on GitHub to understand the testing practices followed by them. For the second study, based on the responses from Android developers, they improved the survey questions and resend it to Windows app developers within Microsoft. Both Android and Windows app developers face many challenges such as time constraints, compatibility issues, lack of exposure, cumbersome tools, etc [14].

Linares-Vasquez et al. (2017) analyzed survey responses from 102 open source contributors to Android projects about their practices when performing testing. The survey focused on questions regarding practices and preferences of developers/testers in-the-wild for (i) designing and generating test cases, (ii) automated testing practices, and (iii) perceptions of quality metrics such as code coverage for determining test quality. Analyzing the information gathered from this survey, they compile a body of knowledge to help guide researchers and professionals toward tailoring new automated testing approaches to the need of a diverse set of open source developers [15].

4 RESEARCH METHODOLOGY

Our research is composed of two studies. The first study was conducted for the purpose of applying the ET to apps that have a diverse context and are already available on Google Play, and to analyze whether testers actually explore all possible scenarios that apps may display. The second study of analogous way, also aims to apply ET but in two applications that were developed by a software development company, this study has the objective of applying the ET in order to identify bugs of different levels, that often cannot be revealed using other techniques, for example automated testing. As apps have different OS, use different contexts as well as sensors, connections, peripherals, there are a multitude of mobile devices with different configurations [11], automated testing will not have as comprehensive coverage as TE. We describe these two studies in details in the following two sections.

4.1 First Experimental Study

The first study was conducted with students from the Federal University of Technology - Parana (UTFPR); they were in the fifth semester of an undergraduate degree in Software Engineering, and were taking the Software Testing course. The study was carried out with forty students (called as participants). Participants were chosen because they already have a more in-depth knowledge of programming and Software Testing. All participants performed ET in different apps in order to find bugs. After performing all tests following the provided instructions, they made use of a checklist to identify if they have been explored potentially-interesting scenarios of their app.

Each participant received a mobile app. The chosen apps were randomly selected based on a list of available open source apps [23]. We selected apps from different contexts, for example, using GPS, using a memory card, accelerometer. Assorted apps have been chosen to cover as many mobile-specific events as possible. Table 1 shows the apps that were tested, rating, and number of downloads. The study was conducted with 40 students, so 40 different apps were tested.

All participants took part in a training about ET. In addition, a road map was drawn containing all the necessary concepts such as mind maps, how to create and use them, and a step by step guide of what they should do. To improve students' understanding of the Application Under Test (AUT), they did create an App's mind map showing all its features that could be tested. It was also a way to get participants to focus as much as possible on exploring their apps to find bugs.

In order to find bugs, participants did use the mental map that facilitated the App's understanding and consequently facilitated the charters creation. After ET completed, participants used a specific checklist for mobile device events to validate whether they actually explored some of the app's interesting features.

The study was divided into some parts. The complete steps that were performed were as follows: (1) First of all participants were given instructions on how to conduct the study, and also a road map containing all the necessary information. (2) The students signed the consent form and then went through an evaluation regarding the information given about ET. (3) The experiment was performed offline; the participants performed all the tests at home. It was given two weeks for them to perform all that was requested for this study, like described in the road map. They explored the App to understand it and created a mind map. (4) At the beginning of tests, the participant dedicated 30 minutes per session. Each session could contain 4 to 5 features. Each participant tested exactly three sessions. During the tests, participants recorded the screen for the

time specified and created a report with all found bugs by each test session. (5) Then, participants answered a questionnaire about the App to be tested. (6) Later, according to the answers of the previous questionnaire, a new checklist for testing was presented to the student. An example of the questionnaire is shown in Figure 2. (7) The participants then submitted their reports and their screen recording, and the bugs identified. In addition to all the checklist responses were collected. (8) Finally, the bugs were confirmed, by the bug report generated by each participant, and the data were summarized.

Questions in the checklist (step 6) were oriented by the Apps characteristics. For example, if an app does not need a connection to the network it was not necessary to check the Apps behavior without connection. However, some questions were generic because of several events may occur on any type of mobile device.

The checklist was defined by one of the authors from the elicitation of mobile-specific events. First, the events were surveyed from 108 papers identified in a systematic mapping. The second step was to verify the identified events through a Test Criteria document for mobile apps in order to improve and confirm the checklist. The document used is developed by AQuA (App Quality Alliance) [1]. This document provides a series of test scenarios that tester should consider when testing an Android app. As a final step, we validate the checklist with IT professionals that work with mobile apps. On average, 80% of the professionals agreed totally or partially with the events found.

The questionnaire has 9 questions that tester should respond according to the app. All questions must be answered, otherwise it will not be possible to finalize the questionnaire to generate the checklist. Some questions use a check boxes, so the tester can check various options as long as he has not checked the "do not use" option. If he have marked this option, all others will be disabled. For each selected option, a series of scenarios are considered in the checklist. Finally, when the checklist is generated the user puts his name and the name of the project or app that has been tested.

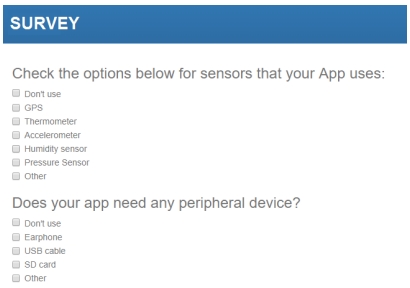

Figure 2: Example of Checklist questionnaire.

4.2 Second Experimental Study

A study was carried out to evaluate the ET technique in applications developed by the Industry. For this purpose, a meeting was held at a development company to provide some of their developed apps so that they could be tested. We were hoping to find flaws that were not revealed previously by the company team. This study was performed by one of the authors.

Initially, a software development company was contacted to set up a meeting for the purpose of presenting the project idea. The meeting was scheduled and all steps and objectives of the project were clarified. The company accepted to participate in the study and made available two applications developed by them. The apps were APP1 and APP2. APP1 is an agenda that allows scheduling tasks by placing day priorities, it is possible insert comments, photos, attachments and tags. Tasks can be ordered by date, by priorities, by title and by the person in charge. Also it can search for text, move tasks to other projects, etc. There is a small menu in the app that shows Inbox Tasks, Today, the Next 7 Days, Delayed, Projects, Labels, and Filters.

APP2 is an app for scheduling audits that can be done in a company. To add an audit you need to create it and add it via a Web platform, the app shows only audits that have been started. The initial screen displays all audits that were previously selected, then the audit can be completed. It can be attributed to a "passed", "must improve", or "failed" audit. If you have only one item disapproved the audit can not be completed. Every insertion, choice of type of audit and where it will be applied is done by the Web, not being part of the app.

The same steps that were followed by the participants of the first study was conducted in the second study by one of the authors. However, all app functionalities were explored and tested. In this study the amount of functionalities that should be tested by charter was not delimited, that is, the whole app should be tested. Both applications were split into charters that would open some features of the app. Some charters even took about 90 minutes, equating like a long charter.

5 ANALYSIS OF RESULTS

5.1 First Study

After completing all tests, each participant submitted: a brief description of the app tested, a bug report, the mental app map, the charter with tested features, the list of tests performed, found bugs, and the videos with the recordings of each session. All bugs were confirmed by one of the authors. Firstly, the report was read and identified each bug; then every bug was confirmed if it was a bug, a false positive or a misconception. Finally the bugs were confirmed for each app through recordings. We replicated the same bug reported by each participant, step by step was done with all the reports delivered.

The Table2 shows the general bugs data collected in this experiment. Among the 40 students, 195 bugs were reported, while 210 bugs were confirmed. This difference is due that some participants reported more than one bug but they counted as only one, since it was found in the same test. Thus, the bugs confirmed were separated and counted as different bugs, therefore totaling the 210 bugs. On average each participant detected 5.2 bugs in 1h30. In addition, 26 false-positives were reported, which are bugs reported by the participants but were discarded for not being bugs, or for not being able to reproduce.

Table 3 shows the number of apps according to specific events reported by participants. For example, the events that was most used out of 40 apps were Network connection and Screen rotation with 31 and 26 Apps respectively.

Table 1: Apps used for first study

App	Rating	Downloads	Category	App	Rating	Downloads	Category
File manager	4,8	10.000.000+	Productivity	c:geo	4,4	5.000.000+	Entertainment
My Library	4,7	1.000.000+	Productivity	EP Mobile	4,4	50.000+	Medicine
reddit is fun (unofficial)	4,7	5.000.000+	News & magazines	GnuCash	4,4	100.000+	Business
ComicScreen - ComicViewer	4,7	1.000.000+	Humor	InstaMag	4,3	10.000.000+	Photography
Editor - InShot	4,7	100.000.000+	Photography	OI Shopping list	4,3	1.000.000+	Shopping
Bangla Dictionary	4,6	5.000.000+	Books & references	QuickLyric	4,3	1.000.000+	Music & Audio
File Manager (File Explorer)	4,6	100.000.000+	Tools	RunnerUp	4,3	10.000+	Health & Fitness
Tram Hunter	4,6	100.000+	Tourism	Subsonic Music Streamer	4,3	500.000+	Music & Audio
AntennaPod	4,6	100.000+	Video players	PhotoPhase	4,3	10.000+	Customization
Mobills	4,6	5.000.000+	Business	OSMTracker	4,2	10.000+	Tourism
Mysplash	4,6	50.000+	Customization	Birthdays into Calendar	4,2	10.000+	Productivity
TED	4,6	10.000.000+	Education	Wordpress	4,2	10.000.000+	Productivity
Daily Money	4,5	500.000+	Business	Reminders	4,2	100.000+	Corporate
Alarm Clock	4,5	50.000.000+	Productivity	Wheelmap	4,2	10.000+	Tourism
AnkiDroid Flashcards	4,5	1.000.000+	Education	OsmAnd	4,2	5.000.000+	Tourism
FBReader	4,5	10.000.000+	Books & references	PocketBand	4,1	1.000.000+	Music & Audio
WiFi Analyzer	4,5	1.000.000+	Tools	Car Cast Podcast Player	4,1	100.000+	Music & Audio
Retroboy	4,5	10.000+	Photography	Notepad - Text Editor	3,9	1.000.000+	Tools
OpenKeychain: Easy PGP	4,4	100.000+	Communication	Jamendo Music	3,4	500.000+	Music & Audio
My Expenses	4,4	500.000+	Business	Smart Scales	2,7	100.000+	Health & Fitness

Table 3: Events and number of Apps by events

Events	# of Apps	Events	# of Apps
Network connection	31	Vibrate	6
Screen Rotation	26	Shake	3
Scroll	23	Earphone	3
Social network	17	Accelerometer	2
Swype	16	SD card	2
Microphone	14	Gyroscope	2
Photographic camera	12	Bluetooth	2
GPS	6		

Table 2: General Bug data

Reported	Confirmed	Not Accepted
195	210	26

The objective of this study was to identify the test scenarios that the participants thought and if these scenarios could cover all the App's scenarios. Thus, we will show all the checklist questions divided by different test contexts, as well as the percentage of participants who have tested or not the given question. It is observed that in several scenarios a large percentage of students did not consider carrying out the tests, the same happens with the specific events coming from sensors, connections, etc.

Most testers who do not have much experience with tests but do have knowledge on the subject focused only on the features of the App and did not identify several scenarios that should be considered. For example, apps can use different types of events, which are often unexpected and may reveal some kind of bug e.g. interrupting an app when receiving a call, or rotating a screen. These events are not tested most of the time, and bugs are only found when the end user perform this events.

General Questions: figure 3 shows all the general questions that are generated in the checklist regardless of which app is being tested. It can be seen that among the 6 questions the percentage of participants who did not test a given question is greater than 50%. For example, 83% of 40 participants did not test their apps on different screen sizes.

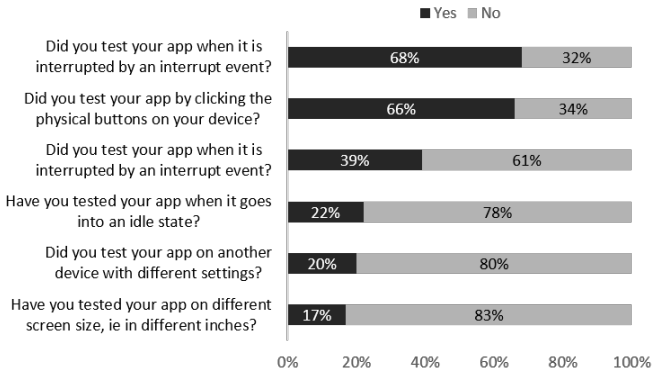

Figure 3: Result of checklist of general questions.

GPS Questions: figure 4 presents questions returned by the checklist in relation to GPS. Table 3 shows us that there were 6 participants who had apps using GPS. Note that participants did not test most of the scenarios listed in the checklist, with the exception of the signal loss, only 33% of participants using GPS in their app did not test this scenario.

Bluetooth Questions: figure 5 presents the questions about the use of Bluetooth; as shown in Table 3, only two participants

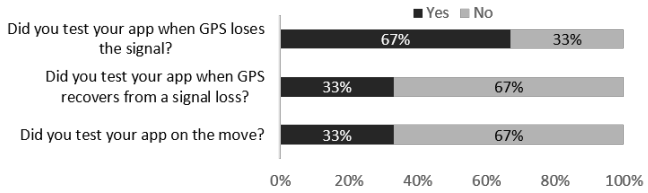

Figure 4: Result of checklist of GPS questions.

tested Bluetooth apps. It can be seen from Figure 5 that most of the scenarios suggested by the checklist were not part of the participants' tests. However the two participants tested their apps with the Bluetooth turned off, which would be more common to happen.

Camera Questions: figure 6 presents the questions about the use of the camera, as shown in Table 3, 12 participants tested apps that used the camera. Unlike the other test groups that the checklist consider, in this case most of the scenarios were tested by the participants. Of 4 questions, only the question of testing camera options was ignored by more than 50% of individuals.

Accelerometer Questions: figure 7 presents the questions about the use of the accelerometer sensor; as shown in Table 3, 2 participants tested apps that made use of the accelerometer. For this context the checklist brings 2 suggestions. The first question was considered by all participants, and the second question 50% considered the scenario and 50% did not consider.

Network Connection Questions: figure 8 presents the questions about the use of the network connection; as shown in table 3, 31 apps made use of network connection. For this context the checklist brings 7 test suggestions. Most scenarios were ignored by participants. The least-tested scenario was exploring the app when it encountered airplane mode, only 13% of participants consider it. The only scenario that got the highest percentage of test was to test the app without the connection, 65% of the participants performed this test.

Earphone Questions: figure 9 presents questions returned by the checklist regarding the use of headphones. Table 3 shows us that there were 3 participants who had apps using earphone. Unlike the other scenarios, the participants tested all suggested scenarios in the checklist, that is, 100% of participants considered the three checklist questions. What has not happened in other contexts.

Microphone Questions: for the questions related to the use of microphone the scenarios presented a more relevant percentage being above 60%, as can be observed in Figure 10 since most of the participants tested the scenarios, except for the question Microphone Options Test, only 21% of participants tested this scenario. According to Table 3, 14 apps used microphone.

Social Network Questions: figure 11 presents the questions about the use of Social Network; as shown in Table 3, 17 apps made use of Social Network. For this context the checklist brings 2 test suggestions. Both scenarios had opposite results. 82% tested apps without connecting to any social networks, and only 18% of respondents tested apps by logging into different accounts.

Screen Rotation Questions: figure 12 presents the questions about the use of screen rotation, as shown in Table 3, 26 participants tested apps that made use of screen rotation. Unlike the other test

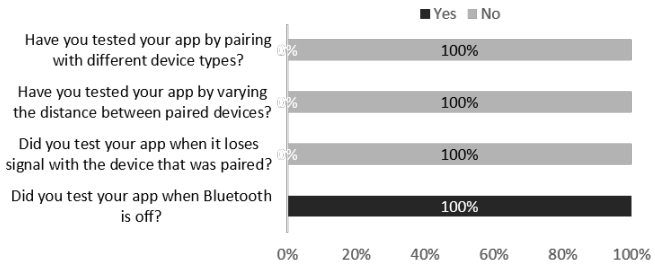

Figure 5: Result of checklist of Bluetooth questions.

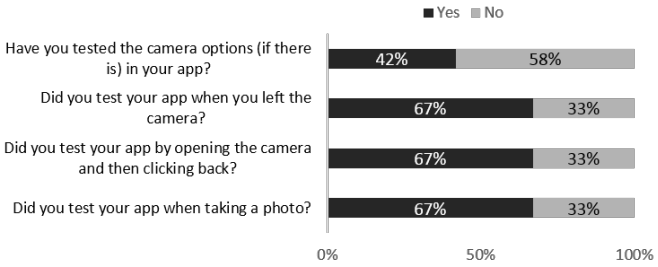

Figure 6: Result of checklist of Camera questions.

groups that the checklist considers, in this case most of the scenarios were tested by the participants. Of 4 questions, only the question of testing screen rotation options was ignored by more than 50% of individuals. For this context the checklist bring 2 questions was both well divided, but both of the questions had the percentage of participants who considered the scenarios greater than 50%.

Scroll Questions: for questions related to the use of Scroll the checklist brings 3 scenarios. Two of them presented a high percentage, both 91% of participants consider such scenarios in the tests, as can be seen in Figure 13. The third scenario also scored a sizable percentage of test, but not as high as the other two, 52% took the test. According to the table 3, 23 apps used scroll.

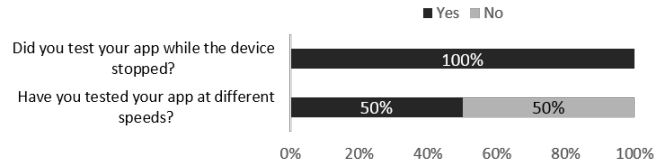

Figure 7: Result of checklist of Accelerometer questions.

Memory Card Questions: figure 14 presents the questions about memory card usage, as shown in Table 3, 2 participants tested apps that made use of the memory stick. For all the questions pointed out by the checklist for such context all participants did not perform any tests considering the use of the memory card. 100% of the participants did not test any of the scenarios pointed out by the checklist.

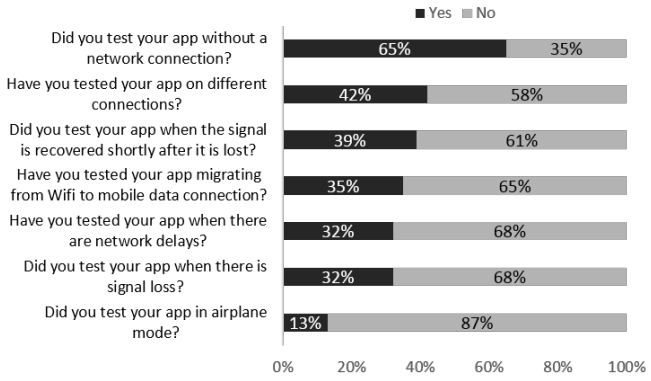

Figure 8: Result of checklist of Network Connection questions.

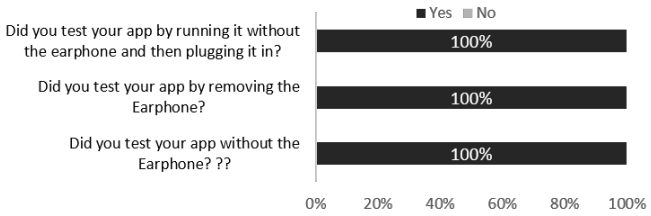

Figure 9: Result of checklist of Earphone questions.

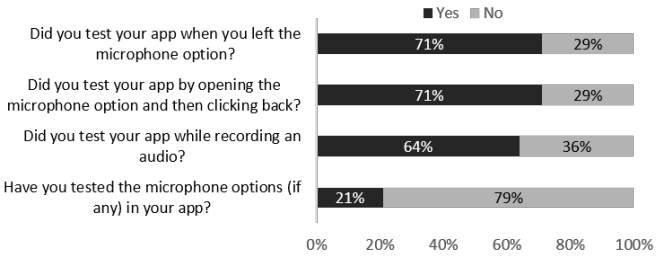

Figure 10: Result of checklist of Microphone questions.

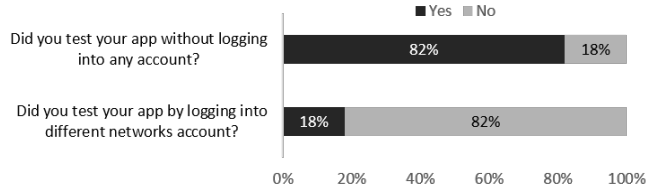

Figure 11: Results of checklist of Social Network questions.

Shake Questions: figure 15 presents the questions about shake usage; as shown in Table 3, 3 participants tested apps that used shake. the checklist considers two scenarios for this context. The result stop both questions was 100%, i.e. all participants tested the two questions pointed out by the checklist.

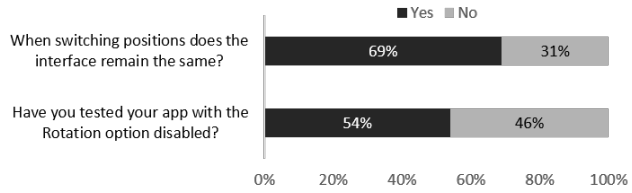

Figure 12: Results of checklist of Screen Rotation questions.

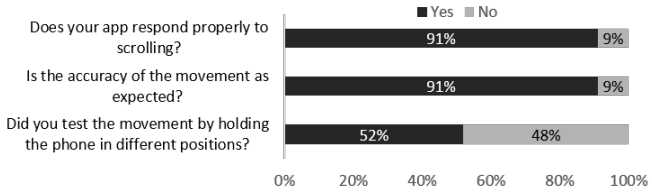

Figure 13: Results of checklist of Scroll questions.

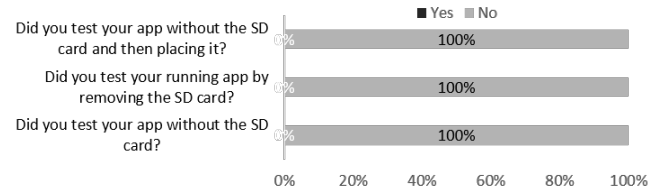

Figure 14: Results of checklist of Memory Card questions.

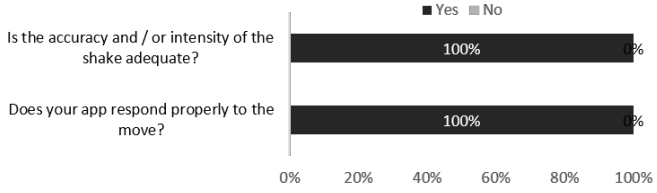

Figure 15: Results of checklist of Shake questions.

Swype Questions: for questions related to using Swype the checklist brings 3 scenarios. All of them presented a high percentage of participants who considered such scenarios in the tests, as can be seen in Figure 16, for one of the scenarios 100% of the participants performed the test if the Swype movement was occurring as expected. The other two scenarios had 62% and 87% participants who performed the tests. According to Table 3 16 apps used microphone.

Vibrate Questions: for issues related to using the Vibrate checklist brings 2 scenarios. Both presented a 83% of tests considered in the scenarios in question. It is possible to check in figure 17 such data. According to table 3 6 apps have used the vibration in their apps.

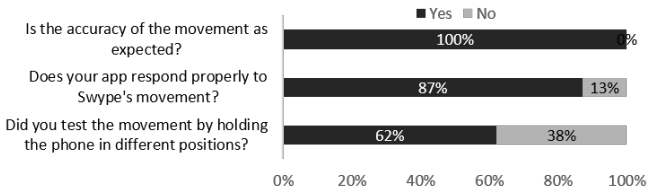

Figure 16: Results of checklist of Swype questions.

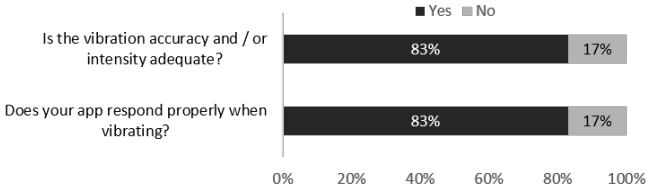

Figure 17: Results of checklist of Vibrate questions.

It's possible to realize that certain test scenarios are overlooked by most participants, leading us to conclude that testers may act the same way. There are quite specific questions they are not reminded to explore, such as testing the app when airplane mode is enabled, or consider Scenarios with Memory Card.

5.2 Second Study

By exploring the applications, defects have been revealed as well as some issues. Table 4 summarizes the number of charters that each app was split, the time of tests, and the number of bugs and issues found.

After testing the two applications and identifying the bugs, a new meeting was arranged with the person representing the company. We presents all the encountered bugs and issues during the ET process. Then, The person representing the company confirmed if the bugs were really bugs or not.

Among the revealed bugs we identified some high criticality bugs, which directly disrupts the use of the app, or some functionalities, and others not so critical. It was also possible to measure the level of difficulty of revealing faults. Table 5 shows which test was done, the bug that revealed, Level of difficulty to reveal the bug, and the criticality of it.

The criticality is presented on a scale of 0 to 10. The closer to zero the value is lower the criticality, and the closer the value is to 10 the higher the criticality. The difficulty level was classified as: Easy, Medium, Hard and very difficult.

Table 4: Results from App1 and App2

	APP1	APP2
Number of Charters	4	1
Total Test Time:	186 min	38 min
Total Bugs Founded	55	9
Total Issues Founded	15	6

6 DISCUSSION AND LESSONS LEARNED

Both studies have some limitations that may be overcome in future with replications and more formal and controlled settings. In the first study, the participants were students and may not represent how professional testers would perform. The second study was conducted with apps from only one software development company. As in the first study, we cannot generalize the results.

Based on the first study, we can observe that there are many specific scenarios of mobile devices that most participants have not tested. Most of the tests carried out by the participants were tests that they placed inputs to verify the expected output, but most did not test other scenarios, such as sensor examples, special events, the use of the camera, headphones etc; that is, did not properly exploit the app.

It's noticed that there are many scenarios of tests that could be explored, and that often the bug can be revealed in these cases, as study II presented. Much of the Bugs in study II were found because we explored different contexts that apps 1 and 2 use, we explored unexpected behaviors, such as clicking an icon repeatedly. Most of the bugs found were not revealed by the company that developed it. With these results, we may concluded that the behavior adopted by participants (study I) is the same as most testers (Study II). ET was very efficient in detecting bugs in study II. This is because exploration is carried out as the literature has proposed, dedicating purely testing effortlessly, using the testator's imagination, and his experience. Based on these results we conclude that ET is an effective approach to bug reporting in mobile apps.

7 THREATS TO VALIDITY

In the following, we identified some threats to **Internal Validity** related to the training (study I). We minimize it validity using procedures and materials with all the necessary steps and concepts that participants should know. Threats to **External Validity** relate to the generalizability of our results, academic students performed the first study and on the second study was used two mobile apps developed by a company. Participants can be considered representative because they have knowledge of mobile application testing and the applications (artifacts) used were developed by a company that operates in the market. A threat to **Conclusion Validity** was performed using the proposed checklist, which identified which test scenarios were omitted.

8 CONCLUSION

In this paper, we conducted two exploratory studies. The former, was performed with 40 participants and 40 mobile apps. The latter, was performed with two mobile apps developed by the Industry. Based on both studies ET has proven to be very good at discovering bugs although test professionals need to better exploit their applications and search for bugs in scenarios related to specific mobile device events. Because as expected there are several mobile device-specific scenarios that testers do not mind and end up not testing such features. In addition, with the use of ET, it was possible to identify bugs of different levels of revelation and of criticality. So we concluded that ET proved to be effective for bug reporting in apps.

Table 5: Bugs App1 and App2

Bug	Level	Criticality	Bug	Level	Criticality	Bug	Level	Criticality
> > > > > APP1 < < < < <								
Charter I								
I-B1	Easy	7	I-B8	Easy	2	I-B15	Easy	4
I-B2	Easy	10	I-B9	Very difficult	2	I-B16	Very difficult	10
I-B3	Easy	10	I-B10	Difficult	5	I-B17	Medium	1
I-B4	Easy	4	I-B11	Difficult	7	I-B18	Easy	2
I-B5	Easy	7	I-B12	Easy	7	I-B19	Very difficult	10
I-B6	Medium	10	I-B13	Difficult	10	I-B20	Difficult	10
I-B7	Easy	4	I-B14	Very difficult	10	I-B21	Medium	10
Charter II								
II-B1	Easy	7	II-B6	Easy	7	II-B10	Very difficult	10
II-B2	Medium	10	II-B7	Difficult	3	II-B11	Difficult	8
II-B3	Medium	6	II-B8	Easy	9	II-B12	Difficult	4
II-B4	Easy	10	II-B9	Difficult	4	II-B13	Difficult	9
II-B5	Easy	10						
Charter III								
III-B1	Easy	5	III-B8	Easy	5	III-B14	Medium	10
III-B2	Easy	8	III-B9	Medium	9	III-B15	Very difficult	8
III-B3	Easy	10	III-B10	Medium	4	III-B16	Medium	10
III-B4	Easy	6	III-B11	Easy	5	III-B17	Very difficult	8
III-B5	Easy	10	III-B12	Very difficult	10	III-B18	Medium	10
III-B6	Medium	1	III-B13	Medium	10	III-B19	Difficult	10
III-B7	Very difficult	10						
Charter IV								
IV-B1	Easy	8	IV-B2	Difficult	7			
> > > > > APP2 < < < < <								
Charter I								
I-B1	Very difficult	10	I-B4	Easy	8	I-B7	Very difficult	9
I-B2	Very difficult	10	I-B5	Easy	8	I-B8	Very difficult	7
I-B3	Medium	6	I-B6	Easy	9	I-B9	Easy	6

Some results (Figures 3 and 4) showed a poor quality of testing process performed. These observed results are interesting because it could be used for future work to improve a test subject. For example, the use of guides or checklists for teaching exploratory testing.

ACKNOWLEDGMENTS

The authors thank the partner IT company that provided the industrial apps used in this study. Special thanks to the participants who took part in the experiments. The authors are also grateful to the anonymous reviewers for their useful comments and suggestions. Andre T. Endo is partially financially supported by CNPq/Brazil (grant number 420363/2018-1). Isabel K. Villanes is supported by CAPES/Brazil.

REFERENCES

[1] App Quality Alliance. 2018. AQuA Performance Testing Criteria. *Retrieved from App Quality Alliance: http://www. appqualityalliance. org/aqua-performance-test-criteria* (2018).

[2] Domenico Amalfitano, Vincenzo Riccio, Ana CR Paiva, and Anna Rita Fasolino. 2018. Why does the orientation change mess up my Android application? From GUI failures to code faults. *Software Testing, Verification and Reliability* 28, 1 (2018), e1654.

[3] James Bach. 2003. Exploratory testing explained.

[4] Jerry Gao, Xiaoying Bai, Wei-Tek Tsai, and Tadahiro Uehara. 2014. Mobile Application Testing: A Tutorial. *Computer* 47, 2 (2014), 46–55. https://doi.org/10.1109/MC.2013.445

[5] Ceren \cSahin Gebizli and Hasan Sözer. 2017. Automated refinement of models for model-based testing using exploratory testing. *Software Quality Journal* 25, 3 (2017), 979–1005. https://doi.org/10.1007/s11219-016-9338-2

[6] Hyung Kil Ham and Young Bom Park. 2011. Mobile application compatibility test system design for Android fragmentation. *Communications in Computer and Information Science* 257 CCIS (2011), 314–320. https://doi.org/10.1007/978-3-642-27207-3_33

[7] Juha Itkonen et al. 2011. Empirical studies on exploratory software testing. (2011).

[8] Juha Itkonen, Mika V Mantyla, and Casper Lassenius. 2007. Defect detection efficiency: Test case based vs. exploratory testing. In *First International Symposium on Empirical Software Engineering and Measurement (ESEM 2007)*. IEEE, 61–70.

[9] Juha Itkonen and Kristian Rautiainen. 2005. Exploratory testing: a multiple case study. In *2005 International Symposium on Empirical Software Engineering, 2005.* IEEE, 10–pp.

[10] K. Rautiainen J. Itkonen. 2005. Exploratory testing: a multiple case study. In *International Symposium on Empirical Software Engineering*. 84–93. https://doi.org/10.1109/ISESE.2005.1541817

[11] Mona Erfani Joorabchi, Ali Mesbah, and Philippe Kruchten. 2013. Real challenges in mobile app development. In *Empirical Software Engineering and Measurement, 2013 ACM/IEEE International Symposium on*. IEEE, 15–24.

[12] Cem Kaner, James Bach, and Bret Pettichord. 2008. *Lessons learned in software testing*. John Wiley & Sons.

[13] Cem Kaner, Jack Falk, and Hung Quoc Nguyen. 1999. Testing Computer Software.

[14] P. S. Kochhar, F. Thung, N. Nagappan, T. Zimmermann, and D. Lo. 2015. Understanding the Test Automation Culture of App Developers. In *2015 IEEE 8th*

International Conference on Software Testing, Verification and Validation (ICST). 1–10. https://doi.org/10.1109/ICST.2015.7102609

[15] Mario Linares-Vásquez, Carlos Bernal-Cárdenas, Kevin Moran, and Denys Poshyvanyk. 2017. How do developers test android applications?. In *Proceedings - 2017 IEEE International Conference on Software Maintenance and Evolution, ICSME 2017*. 613–622. https://doi.org/10.1109/ICSME.2017.47 arXiv:1801.06268

[16] James Lyndsay and Neil Van Eeden. 2003. Adventures in session-based testing. *Workroom Productions Ltd. May* 27 (2003).

[17] Bakhtiar M. Amen, Sardasht M. Mahmood, and Joan Lu. 2015. Mobile Application Testing Matrix and Challenges. In *Computer Science & Information Technology (CS & IT)*. 27–40. https://doi.org/10.5121/csit.2015.50403

[18] Henry Muccini, Antonio Di Francesco, and Patrizio Esposito. 2012. Software testing of mobile applications: Challenges and future research directions. In *Proceedings of the 7th International Workshop on Automation of Software Test*. IEEE Press, 29–35.

[19] Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. 2004. *The art of software testing*. Vol. 2. Wiley Online Library.

[20] Roger S Pressman. 2010. Software Engineering: A Practitioner's Approach, 7/e, RS Pressman & Associates. *Inc., McGraw-Hill, ISBN* 73375977 (2010).

[21] Rudolf Ramler, Georg Buchgeher, and Claus Klammer. 2018. Adapting automated test generation to GUI testing of industry applications. *Information and Software Technology* 93 (2018), 248–263. https://doi.org/10.1016/j.infsof.2017.07.005

[22] Reinaldo Costa Santana. 2008. Computação móvel, histórico da evolução. *Acesso em* 11 (2008).

[23] Davi Bernardo Silva, Marcelo Medeiros Eler, Vinicius HS Durelli, and Andre Takeshi Endo. 2018. Characterizing mobile apps from a source and test code viewpoint. *Information and Software Technology* 101 (2018), 32–50.

[24] Statista. 2019. Number of apps available in leading app stores as of 1st quarter 2019. https://www.statista.com/statistics/276623/number-of-apps-available-in-leading-app-stores/

[25] Andy Tinkham and Cem Kaner. 2003. Learning styles and exploratory testing. In *Proceedings of the Pacific northwest software quality conference*. 00–00. https://doi.org/10.5121/csit.2015.50403

[26] Anthony I Wasserman and Fosser. 2010. Software Engineering Issues for Mobile Application Development. In *FoSER '10 Proceedings of the FSE/SDP workshop on Future of software engineering research*. 397–400. https://doi.org/10.1145/1882362.1882443

[27] James A Whittaker. 2009. *Exploratory software testing: tips, tricks, tours, and techniques to guide test design*. Pearson Education.

[28] James A. Whittaker. 2009. *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design* (1st ed.). Addison-Wesley Professional.

[29] Samer Zein, Norsaremah Salleh, and John Grundy. 2016. A systematic mapping study of mobile application testing techniques. *Journal of Systems and Software* 117 (2016), 334–356.